

TCC805x MCU BSP

API Specification for System Adaptation Layer

Rev. 1.02 [A]

2021-06-01

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	4
1.1	System Adaptation Layer	4
1.2	SAL Interface	4
1.3	OS Porting Interface	4
2	Terms and Abbreviations	5
3	Source Files	6
3.1	Location of Source Files	6
3.2	Simple Description of Source Files	6
4	Types	7
4.1	Primitive Types	7
4.2	Address Types	9
4.3	Handles	10
5	Constants	11
5.1	NULL Values	11
5.2	Boolean Values	12
5.3	SAL Resources	13
5.4	File Attributes	14
6	Enumerations	15
6.1	SALDriverId_t	15
6.2	SALFileMode_t	16
6.3	SALFileSeekOrigin_t	17
6.4	SALBlockingOption_t	18
6.5	SALEventOption_t	19
6.6	SALTaskPriority_t	20
6.7	SALRetCode_t	21
6.8	SALErrorCode_t	22
7	Structures	24
7.1	SALMcuVersionInfo_t	24
7.2	SALErrorInfo_t	25
7.3	SALCoreFunc_t	26
7.4	SALOSRunFunc_t	27
7.5	SALMemFunc_t	28
7.6	SALDataStructFunc_t	29
7.7	SALSyncObjectFunc_t	30
7.8	SALTaskFunc_t	31
7.9	SALTraceFunc_t	32
7.10	SALetcFunc_t	33
8	Macros	34
8.1	Accessing Registers	34
9	Callback Functions	35
9.1	SALTaskFunc	35
10	APIs	36
10.1	SAL_CoreMB	36
10.2	SAL_CoreRMB	37
10.3	SAL_CoreWMB	38
10.4	SAL_CoreCriticalEnter	39
10.5	SAL_CoreCriticalExit	40
10.6	SAL_CoreWaitForEvent	41
10.7	SAL_CoreDiv64To32	42
10.8	SAL_OsInitFuncs	43
10.9	SAL_OsStart	44
10.10	SAL_MemAlloc	45
10.11	SAL_MemCalloc	46
10.12	SAL_MemFree	47
10.13	SAL_MemSet	48
10.14	SAL_MemCopy	49
10.15	SAL_MemCmp	50
10.16	SAL_StrCopy	51
10.17	SAL_StrCmp	52
10.18	SAL_StrNCmp	53
10.19	SAL_StrLength	54
10.20	SAL_QueueCreate	55

10.21	SAL_QueueDelete	56
10.22	SAL_QueueGet	57
10.23	SAL_QueuePut	58
10.24	SAL_EventCreate	59
10.25	SAL_EventDelete	60
10.26	SAL_EventSet	61
10.27	SAL_EventGet	62
10.28	SAL_MutexCreate	63
10.29	SAL_MutexDelete	64
10.30	SAL_MutexLock	65
10.31	SAL_MutexUnlock	66
10.32	SAL_SemaphoreCreate	67
10.33	SAL_SemaphoreDelete	68
10.34	SAL_SemaphoreRelease	69
10.35	SAL_SemaphoreWait	70
10.36	SAL_TaskCreate	71
10.37	SAL_TaskDelete	72
10.38	SAL_TaskCurrent	73
10.39	SAL_TaskSleep	74
10.40	SAL_DbgReportError	75
10.41	SAL_GetCurError	76
10.42	SAL_GetTickCount	77
10.43	SAL_HandleTick	78
10.44	SAL_GetSystemTickCount	79
10.45	SAL_IncreaseSystemTick	80
10.46	SAL_GetVersion	81
11	How to use SAL	82
11.1	Register SAL APIs	82
11.2	Initialize SAL	83
11.3	Start OS by using SAL	83
11.4	Report and Handle Error	85
12	References	87
13	Revision History	88
	Rev. 1.02: 2021-06-01	88
	Rev. 1.01: 2021-01-22	88
	Rev. 1.00: 2020-11-12	88

Figures

Figure 3.1	Location of Source Files	6
Figure 11.1	Flowchart of Registering/Initializing SAL APIs	84
Figure 11.2	Flowchart of Reporting/Handling an Error	86

Tables

Table 2.1	Terms and Abbreviation	5
-----------	------------------------------	---

1 INTRODUCTION

This document describes the System Adaptation Layer (hereinafter referred to as SAL) of TCC805x MCU BSP.

1.1 System Adaptation Layer

SAL is adaptation layer to change OS component and system resources. The SAL increases reusability of own solution and supports various types of RTOS (Telechips currently supports uC/OS-III and freeRTOS). All modules use OS (Operating System) function via SAL interface. System software over the RTOS does not need to be modified while the RTOS changes. System software implements the function by using SAL API. The other part of system modules including device driver and user application can use OS function by using SAL APIs. For this, user application module simply uses OS function by including SAL header file (*sal_api.h*).

1.2 SAL Interface

The SAL supports simple system utilities and OS functions such as task, queue, semaphore, and event flags management. Actually, MCU application uses small part of the whole OS function and the OS function used by MCU application is defined in *sal_com.h* or *sal_internal.h* files. The header file contains prototype of SAL APIs, error value, and some configuration definitions.

1.3 OS Porting Interface

This interface is needed for implementation in compliance with each OS. If you want to use uC/OS-III for MCU OS, the capabilities supported by SAL should be implemented as a function supported by uC/OS-III in *sal_ucos_impl.c* (FreeRTOS in *sal_freertos_impl.c*).

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

SAL	System Adaptation Layer
OS	Operating System
MCU	Micro Controller Unit
FIFO	First In First Out
ISR	Interrupt Service Routine

3 SOURCE FILES

3.1 Location of Source Files

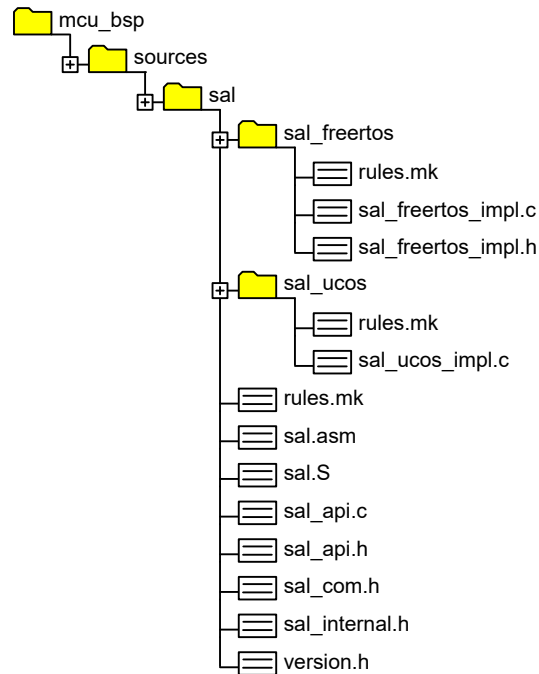


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `sal_[freertos/ucos]_impl.c`
 - The process for initializing SAL is implemented.
 - The required SAL APIs compatible with a ported OS are implemented.
- `sal.asm/S`
 - Core Exception handlers
- `sal_api.c/h`
 - Unique indexes for SAL APIs
 - Functions to initialize SAL APIs
- `sal_com.h`
 - SAL APIs is based on this header file.
 - All of enumerations, constants (macro), structs, and API function table
- `sal_internal.h`
 - Defined as function macro which is easy to invoke SAL APIs.
- `version.h`
 - Version information of MCU BSP

4 TYPES

4.1 Primitive Types

Primitive Type, AUTOSAR type, MISRA C:2012 - 4.6

Declaration

```
typedef char          int8;
typedef signed char   sint8;
typedef unsigned char uint8;
typedef short         int16;
typedef signed short  sint16;
typedef unsigned short uint16;
typedef long          int32;
typedef signed long   sint32;
typedef unsigned long uint32;
typedef long long     int64;
typedef signed long long sint64;
typedef unsigned long long uint64;

typedef uint32        uintptr;
typedef sint32        sintptr;

typedef unsigned char  boolean;

typedef uint32         SALSize;
```

Description

Value	Description
<i>int8</i>	-128 to 127
<i>sint8</i>	-128 to 127
<i>uint8</i>	0 to 255
<i>int16</i>	-32768 to 32767
<i>sint16</i>	-32768 to 32767
<i>uint16</i>	0 to 65535
<i>int32</i>	-2147483648 to 2147483647
<i>sint32</i>	-2147483648 to 2147483647
<i>uint32</i>	0 to 4294967295
<i>int64</i>	-(2 ⁶³) to (2 ⁶³)-1
<i>sint64</i>	-(2 ⁶³) to (2 ⁶³)-1

<i>Value</i>	<i>Description</i>
<i>uint64</i>	<i>0 to 18446744073709551615</i>
<i>uintptr</i>	<i>Unsigned integer pointer type</i>
<i>sintptr</i>	<i>Signed integer pointer type</i>
<i>boolean</i>	<i>Boolean type for use with TRUE/FALSE</i>
<i>SALSize</i>	<i>Size type</i>

Remark

None

4.2 Address Types

Type variables for addresses

Declaration

```
typedef uint32      SALAddr;  
typedef volatile uint32  SALReg32;  
typedef volatile uint64  SALReg64;
```

Description

<i>Value</i>	<i>Description</i>
<i>SALAddr</i>	<i>Default Address type</i>
<i>SALReg32</i>	<i>32-bit Address type</i>
<i>SALReg64</i>	<i>64-bit address type</i>

Remark

None

4.3 Handles

Type variables for handlers (Not supported yet)

Declaration

```
typedef uint32    SALFileHandle;  
typedef uint32    SALSearchHandle;  
typedef uint32    SALSockHandle;
```

Description

Value	Description
<i>SALFileHandle</i>	<i>Handler for File I/O (Not supported yet)</i>
<i>SALSearchHandle</i>	<i>Handler for searching File (Not supported yet)</i>
<i>SALSockHandle</i>	<i>Handler for ethernet socket (Not supported yet)</i>

Remark

None

5 CONSTANTS

5.1 NULL Values

Constant values for NULL

Declaration

```
#ifndef NULL
    #define NULL      (0)
#endif

#ifndef NULL_PTR
    #define NULL_PTR  ((void *)0)
#endif
```

Description

Value	Description
NULL	Meaning of nothing or zero
NULL_PTR	It must be a null pointer value.

Remark

None

5.2 Boolean Values

Constant values for true or false

Declaration

```
#ifndef FALSE
    #define FALSE      (0U)
#endif

#ifndef TRUE
    #define TRUE       (1U)
#endif

#define SALDisabled   (FALSE)
#define SALEnabled    (TRUE)
```

Description

Value	Description
FALSE	False
TRUE	True
SALDisabled	Disabled flag
SALEnabled	Enabled flag

Remark

None

5.3 SAL Resources

There is a definition for determining the number of SAL resources to be used in *sal_com.h* as below. You need to modify the configuration value that sets the number of OS resources to support your system.

Declaration

```
#define SAL_MAX_QUEUE    (32)
#define SAL_MAX_TASK     (32)
#define SAL_MAX_SEMA     (32)
#define SAL_MAX_EVENT    (32)
```

Description

Value	Description
SAL_MAX_QUEUE	The maximum number of Queue
SAL_MAX_TASK	The maximum number of Task
SAL_MAX_SEMA	The maximum number of Semaphore
SAL_MAX_EVENT	The maximum number of Event

Remark

None

5.4 File Attributes

Constant value for attributes of file (Not supported yet).

Declaration

```
#define SAL_FILE_ATTR_NORMAL      (0x00U)
#define SAL_FILE_ATTR_RDONLY     (0x01U)
#define SAL_FILE_ATTR_HIDDEN     (0x02U)
#define SAL_FILE_ATTR_SYSTEM     (0x04U)
#define SAL_FILE_ATTR_SUBDIR     (0x10U)
#define SAL_FILE_ATTR_ARCH       (0x20U)
```

Description

Value	Description
<i>SAL_FILE_ATTR_NORMAL</i>	<i>Normal file - No restriction for read/write (Not supported yet)</i>
<i>SAL_FILE_ATTR_RDONLY</i>	<i>Read only file (Not supported yet)</i>
<i>SAL_FILE_ATTR_HIDDEN</i>	<i>Hidden file (Not supported yet)</i>
<i>SAL_FILE_ATTR_SYSTEM</i>	<i>System file (Not supported yet)</i>
<i>SAL_FILE_ATTR_SUBDIR</i>	<i>Subdirectory (Not supported yet)</i>
<i>SAL_FILE_ATTR_ARCH</i>	<i>Archive file (Not supported yet)</i>

Remark

None

6 ENUMERATIONS

6.1 SALDriverId_t

Enumeration variables of identifier for device drivers or modules.

Declaration

```
typedef enum SALDriverIdTable
{
    SAL_DRVID_SAL      = 100,
    SAL_DRVID_MPU      = 300,
    SAL_DRVID_GCI      = 500,
    SAL_DRVID_ISO      = 700,
    SAL_DRVID_MIDF     = 800,
    SAL_DRVID_CAN      = 1000,
    SAL_DRVID_GPSB     = 1100,
    SAL_DRVID_UART     = 1200,
    SAL_DRVID_I2C      = 1300,
    SAL_DRVID_PDM      = 1400,
    SAL_DRVID_ICTC     = 1500,
    SAL_DRVID_GDMA     = 1600,
    SAL_DRVID_ADC      = 1700,
    SAL_DRVID_MBOX     = 1800,
    SAL_DRVID_TMR      = 1900,
    SAL_DRVID_WDT      = 2000,
    SAL_DRVID_CK_C     = 2100,
    SAL_DRVID_GPIO     = 2200,
    SAL_DRVID_PMIO     = 2300,
    SAL_DRVID_DSE      = 2500,
    SAL_DRVID_SDM      = 2600,
    SAL_DRVID_FMU      = 3000,
    SAL_DRVID_FWUG     = 3100,
    SAL_DRVID_LIN      = 3200
} SALDriverId_t;
```

Description

Value	Description
SAL_DRVID_XXX	Identifier of each driver. It is used to inform users of debug information or to use for debugging.

Remark

None

6.2 SALFileMode_t

Enumeration variables for file mode. (Not supported yet)

Declaration

```
typedef enum SALFileMode
{
    SAL_FILE_READ      = 1,
    SAL_FILE_WRITE     = 2,
    SAL_FILE_READWRITE = 3,
    SAL_FILE_CREATE    = 4
} SALFileMode_t;
```

Description

Value	Description
<i>SAL_FILE_READ</i>	<i>Mode of reading the file (Not supported yet).</i>
<i>SAL_FILE_WRITE</i>	<i>Mode of writing the file (Not supported yet).</i>
<i>SAL_FILE_READWRITE</i>	<i>Mode of reading/writing the file (Not supported yet).</i>
<i>SAL_FILE_CREATE</i>	<i>Mode of creating a file (Not supported yet).</i>

Remark

None

6.3 SALFileSeekOrigin_t

Enumeration variables for the seeking point in the file. (Not supported yet)

Declaration

```
typedef enum SALFileSeekOrigin
{
    SAL_SEEK_BEGIN    = 0,
    SAL_SEEK_CUR      = 1,
    SAL_SEEK_END       = 2
} SALFileSeekOrigin_t;
```

Description

Value	Description
SAL_SEEK_BEGIN	Beginning of file (Not supported yet)
SAL_SEEK_CUR	Current position (Not supported yet)
SAL_SEEK_END	End of file (Not supported yet)

Remark

None

6.4 SALBlockingOption_t

Enumeration variables of blocking options for Queue or Semaphore.

Declaration

```
typedef enum SALBlockingOption
{
    SAL_OPT_BLOCKING      = 0x00000000UL,
    SAL_OPT_NON_BLOCKING  = 0x00008000UL
} SALBlockingOption_t;
```

Description

Value	Description
SAL_OPT_BLOCKING	Blocking option
SAL_OPT_NON_BLOCKING	Non-blocking option

Remark

None

6.5 SALEventOption_t

Enumeration variables to specify which bits to be set or cleared. Each bit set in flags will set the corresponding bit in the event flag group. As of now, ***SAL_EVENT_OPT_FLAG_SET*** and ***SAL_EVENT_OPT_CLR_ALL*** are only available on FreeRTOS/uCOS.

Declaration

```
typedef enum SALEventOption
{
    SAL_EVENT_OPT_FLAG_SET    = 0x00000000UL,
    SAL_EVENT_OPT_CLR_ALL     = 0x00000001UL,
    SAL_EVENT_OPT_CLR_ANY     = 0x00000002UL,
    SAL_EVENT_OPT_SET_ALL     = 0x00000004UL,
    SAL_EVENT_OPT_SET_ANY     = 0x00000008UL,
    SAL_EVENT_OPT_CONSUME     = 0x00000100UL
} SALEventOption_t;
```

Description

Value	Description
<i>SAL_EVENT_OPT_FLAG_SET</i>	<i>Initial state signaled</i>
<i>SAL_EVENT_OPT_CLR_ALL</i>	<i>Initial state non-signaled</i>
<i>SAL_EVENT_OPT_CLR_ANY</i>	<i>Wait for any of the bits specified by CLR</i>
<i>SAL_EVENT_OPT_SET_ALL</i>	<i>Wait for all the bits specified by SET</i>
<i>SAL_EVENT_OPT_SET_ANY</i>	<i>Wait for any of the bits specified by SET</i>
<i>SAL_EVENT_OPT_CONSUME</i>	<i>Reserved (Not supported yet)</i>

Remark

None

6.6 SALTTaskPriority_t

Enumeration variables of task priority. The lower the value, the higher the priority.

Declaration

```
typedef enum SALTTaskPriority
{
    SAL_PRIO_APP_CFG           = 2,
    SAL_PRIO_POWER_MANAGER     = 3,
    SAL_PRIO_IPC_CONTROL       = 3,
    SAL_PRIO_SYSTEM_MON        = 4,
    SAL_PRIO_CAN_DEMO          = 4,
    SAL_PRIO_VSDR_MONITOR      = 5,
    SAL_PRIO_VEHICLE_SIGNAL_DEMO = 6,
    SAL_PRIO_IPC_PARSER        = 6,
    SAL_PRIO_KEY_APP           = 6,
    SAL_PRIO_UART_TEST         = 6,
    SAL_PRIO_CONSOLE_DEMO      = 6,
    SAL_PRIO_NVM_MAMAGER       = 8,
    SAL_PRIO_FWUG_APP          = 10,
    SAL_PRIO_LOWEST            = 12
} SALTTaskPriority_t;
```

Description

Value	Description
<i>SAL_PRIO_APP_CFG</i>	<i>APP TASK START task</i>
<i>SAL_PRIO_POWER_MANAGER</i>	<i>Power Manager APP task</i>
<i>SAL_PRIO_IPC_CONTROL</i>	<i>IPC Control task</i>
<i>SAL_PRIO_SYSTEM_MON</i>	<i>System Monitoring APP task</i>
<i>SAL_PRIO_CAN_DEMO</i>	<i>CAN DEMO APP task</i>
<i>SAL_PRIO_VSDR_MONITOR</i>	<i>Vehicle Signal DEMO APP task</i>
<i>SAL_PRIO_VEHICLE_SIGNAL_DEMO</i>	<i>VSDR Receive Monitor Task</i>
<i>SAL_PRIO_IPC_PARSER</i>	<i>IPC Parser task</i>
<i>SAL_PRIO_KEY_APP</i>	<i>KEY APP task</i>
<i>SAL_PRIO_UART_TEST</i>	<i>UART Test APP task</i>
<i>SAL_PRIO_CONSOLE_DEMO</i>	<i>Console DEMO APP task</i>
<i>SAL_PRIO_NVM_MAMAGER</i>	<i>NVM Manager APP task</i>
<i>SAL_PRIO_FWUG_APP</i>	<i>FWUG APP task</i>
<i>SAL_PRIO_LOWEST</i>	<i>APP TASK START task</i>

Remark

None

6.7 SALRetCode_t

This function is used for the return values of SAL APIs as a result of SAL and is defined in *sal_com.h*.

Declaration

```
typedef enum SALRetCode
{
    SAL_RET_SUCCESS    = 0,
    SAL_RET_FAILED     = 1
} SALRetCode_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Return value for success</i>
<i>SAL_RET_FAILED</i>	<i>Return value for failure</i>

Remark

None

6.8 SALErrorCode_t

Enumeration variables to report the detailed error information.

Declaration

```
typedef enum SALErrorCode
{
    SAL_ERR_INIT                = 100,
    SAL_ERR_NO_SPACE            = 101,
    SAL_ERR_INVALID_PARAMETER   = 102,
    SAL_ERR_NOT_SUPPORT         = 103,
    SAL_ERR_TIMEOUT             = 104,
    SAL_ERR_INVALID_HANDLE     = 105,
    SAL_ERR_NO_DATA             = 106,
    SAL_ERR_UNDEF_STATE         = 107,
    SAL_ERR_FAIL_CREATE         = 108,
    SAL_ERR_FAIL_GET_DATA       = 109,
    SAL_ERR_FAIL_SEND_DATA      = 110,
    SAL_ERR_FAIL_START          = 111,
    SAL_ERR_FAIL_DELETE         = 112,
    SAL_ERR_FAIL_RELEASE        = 113,
    SAL_ERR_UNINIT_ITEM         = 114,
    SAL_ERR_OUTOF_SIZE          = 115,
    SAL_ERR_FAIL_GET_KEY        = 116,
    SAL_ERR_FAIL_SET_CONFIG     = 117,
    SAL_ERR_NOT_USEFUL          = 118
} SALErrorCode_t;
```

Description

Value	Description
SAL_ERR_INIT	Initialization error for each API
SAL_ERR_NO_SPACE	No more space (memory, channel, etc.)
SAL_ERR_INVALID_PARAMETER	Invalid parameter is passed.
SAL_ERR_NOT_SUPPORT	Not supported operation or resources
SAL_ERR_TIMEOUT	Timed out while processing
SAL_ERR_INVALID_HANDLE	Invalid handle is detected.
SAL_ERR_NO_DATA	No data
SAL_ERR_UNDEF_STATE	Not defined state
SAL_ERR_FAIL_CREATE	Fail to create a component (Task, event, etc.)
SAL_ERR_FAIL_GET_DATA	Fail to get data from a component
SAL_ERR_FAIL_SEND_DATA	Fail to send data to a component
SAL_ERR_FAIL_START	Fail to start a component

<i>Value</i>	<i>Description</i>
<i>SAL_ERR_FAIL_DELETE</i>	<i>Fail to delete a job of a component</i>
<i>SAL_ERR_FAIL_RELEASE</i>	<i>Fail to release a job of a component</i>
<i>SAL_ERR_UNINIT_ITEM</i>	<i>Uninitialized item, variable, or contents</i>
<i>SAL_ERR_OUTOF_SIZE</i>	<i>Size is overflow or is lower than threshold.</i>
<i>SAL_ERR_FAIL_GET_KEY</i>	<i>Fail to get key or ownership</i>
<i>SAL_ERR_FAIL_SET_CONFIG</i>	<i>Fail to set configuration or status</i>
<i>SAL_ERR_NOT_USEFUL</i>	<i>The status is not available.</i>

Remark

None

7 STRUCTURES

7.1 SALMcuVersionInfo_t

Structure table including version information of MCU BSP.

Declaration

```
typedef struct SALVersionInfoTable
{
    uint8    viMajorVersion;
    uint8    viMinorVersion;
    uint8    viPatchVersion;
    uint8    viReserved;
} SALMcuVersionInfo_t;
```

Description

Value	Description
viMajorVersion	Make incompatible API changes (Official Release)
viMinorVersion	Add functionality in a backwards compatible manner
viPatchVersion	Fix backwards compatible bug
viReserved	Reserved (Not supported yet)

Remark

None

7.2 SALErrorInfo_t

Structure table including error information. This table is used for monitoring current state of error.

Declaration

```
typedef struct SALErrorInfoTable
{
    SALDriverId_t    eiDrvId;
    uint32           eiApiId;
    SALErrorCode_t   eiErrorId;
} SALErrorInfo_t;
```

Description

Value	Description
<i>eiDrvId</i>	Driver/Module ID. Refer to SALDriverId t.
<i>eiApiId</i>	API function ID
<i>eiErrorId</i>	Error ID. Refer to SALErrorCode t.

Remark

None

7.3 SALCoreFunc_t

Structure table including APIs for ARM Core. After registering the attribute of this structure with *SAL_RegisterFunction()* function, the required function is invoked. However, it is recommended to call the required function in *sal_internal.h* with consideration for null safety.

Declaration

```
typedef struct SALCoreTable
{
    SALCoreMB          fnCoreMB;
    SALCoreRMB         fnCoreRMB;
    SALCoreWMB         fnCoreWMB;
    SALCoreCriticalEnter fnCoreCriticalEnter;
    SALCoreCriticalExit fnCoreCriticalExit;
    SALCoreWaitForEvent fnCoreWaitForEvent;
    SALCoreDiv64To32    fnCoreDiv64To32;
} SALCoreFunc_t;
```

Description

Value	Description
<i>fnCoreMB</i>	Correspond to <i>SAL_CoreMB()</i> . Refer to SAL CoreMB .
<i>fnCoreRMB</i>	Correspond to <i>SAL_CoreRMB()</i> . Refer to SAL CoreRMB .
<i>fnCoreWMB</i>	Correspond to <i>SAL_CoreWMB()</i> . Refer to SAL CoreWMB .
<i>fnCoreCriticalEnter</i>	Correspond to <i>SAL_CoreCriticalEnter()</i> . Refer to SAL CoreCriticalEnter .
<i>fnCoreCriticalExit</i>	Correspond to <i>SAL_CoreCriticalExit()</i> . Refer to SAL CoreCriticalExit .
<i>fnCoreWaitForEvent</i>	Correspond to <i>SAL_CoreWaitForEvent()</i> . Refer to SAL CoreWaitForEvent .
<i>fnCoreDiv64To32</i>	Correspond to <i>SAL_CoreDiv64To32()</i> . Refer to SAL CoreDiv64To32 .

Remark

None

7.4 SALOSRunFunc_t

Structure table including APIs for OS runtime. After registering the attribute of this structure with *SAL_RegisterFunction()* function, the required function is invoked. However, it is recommended to call the required function in *sal_internal.h* with consideration for null safety.

Declaration

```
typedef struct SALOSRunTable
{
    SALOSInitFunctions    fnOsInitFuncs;
    SALOSStart            fnOsStart;
} SALOSRunFunc_t;
```

Description

Value	Description
<i>fnOsInitFuncs</i>	Correspond to <i>SAL_OsInitFuncs()</i> . Refer to SAL_OsInitFuncs .
<i>fnOsStart</i>	Correspond to <i>SAL_OsStart()</i> . Refer to SAL_OsStart .

Remark

None

7.5 SALMemFunc_t

Structure table including APIs for memory. After registering the attribute of this structure with ***SAL_RegisterFunction()*** function, the required function is invoked. However, it is recommended to call the required function in ***sal_internal.h*** with consideration for null safety.

Declaration

```
typedef struct SALMemTable
{
    SALMemAlloc    fnMemAlloc;
    SALMemCalloc   fnMemCalloc;
    SALMemFree     fnMemFree;
    SALMemSet      fnMemSet;
    SALMemCopy     fnMemCopy;
    SALMemCmp      fnMemCmp;
    SALStrCopy     fnStrCopy;
    SALStrCmp      fnStrCmp;
    SALStrNCmp     fnStrNCmp;
    SALStrLength   fnStrLength;
} SALMemFunc_t;
```

Description

Value	Description
<i>fnMemAlloc</i>	Correspond to <i>SAL_MemAlloc()</i> . Refer to SAL_MemAlloc .
<i>fnMemCalloc</i>	Correspond to <i>SAL_MemCalloc()</i> . Refer to SAL_MemCalloc .
<i>fnMemFree</i>	Correspond to <i>SAL_MemFree()</i> . Refer to SAL_MemFree .
<i>fnMemSet</i>	Correspond to <i>SAL_MemSet()</i> . Refer to SAL_MemSet .
<i>fnMemCopy</i>	Correspond to <i>SAL_MemCopy()</i> . Refer to SAL_MemCopy .
<i>fnMemCmp</i>	Correspond to <i>SAL_MemCmp()</i> . Refer to SAL_MemCmp .
<i>fnStrCopy</i>	Correspond to <i>SAL_StrCopy()</i> . Refer to SAL_StrCopy .
<i>fnStrCmp</i>	Correspond to <i>SAL_StrCmp()</i> . Refer to SAL_StrCmp .
<i>fnStrNCmp</i>	Correspond to <i>SAL_StrNCmp()</i> . Refer to SAL_StrNCmp .
<i>fnStrLength</i>	Correspond to <i>SAL_StrLength()</i> . Refer to SAL_StrLength .

Remark

None

7.6 SALDataStructFunc_t

Structure table including APIs for data structure. After registering the attribute of this structure with ***SAL_RegisterFunction()*** function, the required function is invoked. However, it is recommended to call the required function in ***sal_internal.h*** with consideration for null safety.

Declaration

```
typedef struct SALDataStructureTable
{
    SALQueueCreate    fnQueueCreate;
    SALQueueDelete    fnQueueDelete;
    SALQueueGet       fnQueueGet;
    SALQueuePut       fnQueuePut;
} SALDataStructFunc_t;
```

Description

Value	Description
<i>fnQueueCreate</i>	Correspond to <i>SAL_QueueCreate()</i> . Refer to SAL QueueCreate .
<i>fnQueueDelete</i>	Correspond to <i>SAL_QueueDelete()</i> . Refer to SAL QueueDelete .
<i>fnQueueGet</i>	Correspond to <i>SAL_QueueGet()</i> . Refer to SAL QueueGet .
<i>fnQueuePut</i>	Correspond to <i>SAL_QueuePut()</i> . Refer to SAL QueuePut .

Remark

None

7.7 SALSynObjectFunc_t

Structure table including APIs for synchronization object. After registering the attribute of this structure with **SAL_RegisterFunction()** function, the required function is invoked. However, it is recommended to call the required function in **sal_internal.h** with consideration for null safety.

Declaration

```
typedef struct SALSynObjectTable
{
    /* Event */
    SALEventCreate      fnEventCreate;
    SALEventDelete      fnEventDelete;
    SALEventSet         fnEventSet;
    SALEventGet         fnEventGet;

    /* Mutex */
    SALMutexCreate      fnMutexCreate;
    SALMutexDelete      fnMutexDelete;
    SALMutexLock        fnMutexLock;
    SALMutexUnlock      fnMutexUnlock;

    /* Semaphore */
    SALSemaphoreCreate  fnSemaphoreCreate;
    SALSemaphoreDelete  fnSemaphoreDelete;
    SALSemaphoreRelease fnSemaphoreRelease;
    SALSemaphoreWait    fnSemaphoreWait;
} SALSynObjectFunc_t;
```

Description

Value	Description
<i>fnEventCreate</i>	Correspond to SAL_EventCreate() . Refer to SAL_EventCreate .
<i>fnEventDelete</i>	Correspond to SAL_EventDelete() . Refer to SAL_EventDelete .
<i>fnEventSet</i>	Correspond to SAL_EventSet() . Refer to SAL_EventSet .
<i>fnEventGet;</i>	Correspond to SAL_EventGet() . Refer to SAL_EventGet .
<i>fnMutexCreate</i>	Correspond to SAL_MutexCreate() . Refer to SAL_MutexCreate .
<i>fnMutexDelete</i>	Correspond to SAL_MutexDelete() . Refer to SAL_MutexDelete .
<i>fnMutexLock</i>	Correspond to SAL_MutexLock() . Refer to SAL_MutexLock .
<i>fnMutexUnlock</i>	Correspond to SAL_MutexUnlock() . Refer to SAL_MutexUnlock .
<i>fnSemaphoreCreate</i>	Correspond to SAL_SemaphoreCreate() . Refer to SAL_SemaphoreCreate .
<i>fnSemaphoreDelete</i>	Correspond to SAL_SemaphoreDelete() . Refer to SAL_SemaphoreDelete .
<i>fnSemaphoreRelease</i>	Correspond to SAL_SemaphoreRelease() . Refer to SAL_SemaphoreRelease .
<i>fnSemaphoreWait</i>	Correspond to SAL_SemaphoreWait() . Refer to SAL_SemaphoreWait .

Remark

None

7.8 SALTTaskFunc_t

Structure table including APIs for Task. After registering the attribute of this structure with *SAL_RegisterFunction()* function, the required function is invoked. However, it is recommended to call the required function in *sal_internal.h* with consideration for null safety.

Declaration

```
typedef struct SALTTaskTable
{
    SALTTaskCreate      fnTaskCreate;
    SALTTaskCurrent     fnTaskCurrent;
    SALTTaskDelete      fnTaskDelete;
    SALTTaskSleep       fnTaskSleep;
} SALTTaskFunc_t;
```

Description

Value	Description
fnTaskCreate	Correspond to <i>SAL_TaskCreate()</i> . Refer to SAL_TaskCreate .
fnTaskCurrent	Correspond to <i>SAL_TaskCurrent()</i> . Refer to SAL_TaskCurrent .
fnTaskDelete	Correspond to <i>SAL_TaskDelete()</i> . Refer to SAL_TaskDelete .
fnTaskSleep	Correspond to <i>SAL_TaskSleep()</i> . Refer to SAL_TaskSleep .

Remark

None

7.9 SALTraceFunc_t

Structure table including APIs for debugging. After registering the attribute of this structure with *SAL_RegisterFunction()* function, the required function is invoked. However, it is recommended to call required function in *sal_internal.h* with consideration for null safety.

Declaration

```
typedef struct SALTraceTable
{
    SALDgbReportError    fnReportError;
    SALGetCurrentError    fnGetCurrentError;
} SALTraceFunc_t;
```

Description

Value	Description
fnReportError	Correspond to <i>SAL_ReportError()</i> . Refer to SAL DgbReportError .
fnGetCurrentError	Correspond to <i>SAL_GetCurrentError()</i> . Refer to SAL GetCurError .

Remark

None

7.10 SALEtcFunc_t

Structure table including APIs for the others. After registering the attribute of this structure with *SAL_RegisterFunction()* function, the required function is invoked. However, it is recommended to call the required function in *sal_internal.h* with consideration for null safety.

Declaration

```
typedef struct SALEtcTable
{
    SALGetTickCount          fnGetTickCount;
    SALHandleTick           fnHandleTick;
    SALGetVersion            fnGetVersion;
    SALGetSystemTickCount    fnGetSystemTickCount;
    SALIncreaseSystemTick    fnIncreaseSystemTick;
} SALEtcFunc_t;
```

Description

Value	Description
<i>fnGetTickCount</i>	Correspond to <i>SAL_GetTickCount()</i> . Refer to SAL_GetTickCount .
<i>fnHandleTick</i>	Correspond to <i>SAL_HandleTick()</i> . Refer to SAL_HandleTick .
<i>fnGetVersion</i>	Correspond to <i>SAL_GetVersion()</i> . Refer to SAL_GetVersion .
<i>fnGetSystemTickCount</i>	Correspond to <i>SAL_GetSystemTickCount()</i> . Refer to SAL_GetSystemTickCount .
<i>fnIncreaseSystemTick</i>	Correspond to <i>SAL_IncreaseSystemTick()</i> . Refer to SAL_IncreaseSystemTick .

Remark

None

8 MACROS

8.1 Accessing Registers

Macro functions for accessing registers.

Declaration

```
#define SAL_WriteReg(v, a)    (*((volatile uint32 *) (a)) = (uint32)(v))
#define SAL_ReadReg(a)       (*((volatile uint32 *) (a))
```

Description

Value	Description
SAL_WriteReg	Write a value to specific register address.
SAL_ReadReg	Read a value of specific register address.

Remark

None

9 CALLBACK FUNCTIONS

9.1 SALTTaskFunc

Function pointer to the task entry function.

Declaration

```
typedef void (*SALTTaskFunc)
(
    void *    pArg;
);
```

Parameters

Value	Description
pArg	Additional arguments that you want to pass.

Return

None

Remark

None

10 APIs

10.1 SAL_CoreMB

A full system memory barrier. No instruction in program order after this instruction is executed until this instruction is completed. This order is displayed on all bus masters in the system.

Declaration

```
SALRetCode_t SAL_CoreMB
(
    void
);
```

Parameters

None

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.2 SAL_CoreRMB

Similar to *SAL_CoreMB()*, but only guarantees the order between read accesses. No instruction in program order after *SAL_CoreMB()* is executed until *SAL_CoreMB()* is completed.

Declaration

```
SALRetCode_t SAL_CoreRMB  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.3 SAL_CoreWMB

Similar to *SAL_CoreMB()*, but only guarantees the order between the write accesses. No instruction in program order after *SAL_CoreMB()* is executed until *SAL_CoreMB()* is completed.

Declaration

```
SALRetCode_t SAL_CoreWMB  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.4 SAL_CoreCriticalEnter

Disable/Enable interrupts by preserving the state of interrupts. The state of the interrupt disable flag is stored in the local variable 'gCPU_SR', and then the interrupts are disabled ('gCPU_SR' is allocated in all functions that need to disable interrupts). The previous interrupt state is restored by copying 'gCPU_SR' into the CPU's status register.

Declaration

```
SALRetCode_t SAL_CoreCriticalExit
(
    void
);
```

Parameters

None

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.5 SAL_CoreCriticalExit

Disable/Enable interrupts by preserving the state of interrupts. The state of the interrupt disable flag is stored in the local variable 'gCPU_SR', and then the interrupts are disabled ('gCPU_SR' is allocated in all functions that need to disable interrupts). The previous interrupt state is restored by copying 'gCPU_SR' into the CPU's status register.

Declaration

```
SALRetCode_t SAL_CoreCriticalExit
(
    void
);
```

Parameters

None

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.6 SAL_CoreWaitForEvent

This function enters the sleep state and exits when an exception is received.

Declaration

```
SALRetCode_t SAL_CoreWaitForEvent
(
    void
);
```

Parameters

None

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.7 SAL_CoreDiv64To32

This function is to divide 64 bits by 32 bits.

Declaration

```
SALRetCode_t SAL_CoreDiv64To32)
(
    uint64 *    pullDividend,
    uint32      uiDivisor,
    uint32 *    puiRem
);
```

Parameters

Value	Description
<i>pullDividend</i>	<i>[in] Dividend</i>
<i>uiDivisor</i>	<i>[in] Divisor</i>
<i>puiRem</i>	<i>[out] Reminder</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.8 SAL_OsInitFuncs

Some optional functions that are supported by only specific OS.

Declaration

```
SALRetCode_t SAL_OsInitFuncs  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.9 SAL_OsStart

This function is to start OS services including OS scheduler or multitasking.

Declaration

```
SALRetCode_t SAL_OsStart  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.10 SAL_MemAlloc

This function is not supported by RTOS.

Declaration

```
SALRetCode_t SAL_MemAlloc  
(  
    void *    pMem,  
    uint32    uiSize  
);
```

Parameters

Value	Description
<i>pMem</i>	<i>Not supported yet</i>
<i>uiSize</i>	<i>Not supported yet</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.11 SAL_MemCalloc

This function is not supported by RTOS.

Declaration

```
SALRetCode_t SAL_MemCalloc  
(  
    void *    pMem,  
    uint32    uiNum,  
    SALSize   uiSize  
);
```

Parameters

Value	Description
<i>pMem</i>	<i>Not supported yet</i>
<i>uiNum</i>	<i>Not supported yet</i>
<i>uiSize</i>	<i>Not supported yet</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.12 SAL_MemFree

This function is not supported by RTOS.

Declaration

```
SALRetCode_t SAL_MemFree  
(  
    void *    pMem  
);
```

Parameters

Value	Description
<i>pMem</i>	<i>Not supported yet</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.13 SAL_MemSet

This function sets the first *uiSize* bytes of the memory block pointed by *pMem* to the specified value.

Declaration

```
SALRetCode_t SAL_MemSet
(
    void *      pMem,
    uint8       ucValue,
    SALSizet    uiSize
);
```

Parameters

Value	Description
<i>pMem</i>	<i>[in]</i> Pointer to the memory block to fill.
<i>ucValue</i>	<i>[in]</i> Value to be set. The value is passed as an int, but the function fills the memory block using the unsigned char conversion of this value.
<i>uiSize</i>	<i>[in]</i> The number of bytes to be set to the value. Refer to <i>SALSize</i> in Primitive Types .

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

10.14 SAL_MemCopy

This function copies the values of *uiSize* bytes from the location pointed to by source directly to the memory block pointed to by destination.

Declaration

```
SALRetCode_t SAL_MemCopy
(
    void *      pDest,
    const void * pSrc,
    SALSize     uiSize
);
```

Parameters

Value	Description
<i>pDest</i>	[out] Pointer to the destination array where the content is to be copied, type-casted to a pointer of type void*.
<i>pSrc</i>	[in] Pointer to the source of data to be copied, type-casted to a pointer of type const void*.
<i>uiSize</i>	[in] The number of bytes to be copied. Refer to SALSize in Primitive Types .

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.15 SAL_MemCmp

This function compares the block of *pMem1* to the block of *pMem2*.

Declaration

```
SALRetCode_t SAL_MemCmp
(
    const void *   pMem1,
    const void *   pMem2,
    SALSize        uiSize,
    sint32 *        piRetCmp
);
```

Parameters

Value	Description
<i>pMem1</i>	[in] Pointer to memory block.
<i>pMem2</i>	[in] Pointer to memory block.
<i>uiSize</i>	[in] The number of bytes to be compared. Refer to <i>SALSize</i> in the Primitive Types .
<i>piRetCmp</i>	[out] Integral value indicating the relationship between the content of the memory blocks. ■ <0: the first byte that does <u>not</u> match in both memory blocks has a lower value in <i>pMem1</i> than in <i>pMem2</i>. ■ 0: the contents of both memory blocks are equal. ■ >0: the first byte that does <u>not</u> match in both memory blocks has a greater value in <i>pMem1</i> than in <i>pMem2</i>.

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

10.16 SAL_StrCopy

This function copies the C string pointed by source into the array pointed by destination, including the termination of null character

Declaration

```
SALRetCode_t SAL_StrCopy  
(  
    uint8 *      pucDestString,  
    const uint8 * pucSrcString  
);
```

Parameters

Value	Description
<i>pucDestString</i>	<i>[out] Pointer to the destination array where the content is to be copied.</i>
<i>pucSrcString</i>	<i>[in] C string to be copied.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.17 SAL_StrCmp

This function compares the C string *puc1String* to the C string *puc2String*. This function starts comparing the first character of each string.

Declaration

```
SALRetCode_t SAL_StrCmp
(
    const uint8 * puc1String,
    const uint8 * puc2String,
    sint32 *      piRetCmp
);
```

Parameters

Value	Description
<i>puc1String</i>	[in] C string to be compared
<i>puc2String</i>	[in] C string to be compared
<i>piRetCmp</i>	[out] an integral value indicating the relationship between the strings. ■ <0: the first character that does <u>not</u> match has a lower value in <i>puc1String</i> than in <i>puc2String</i>. ■ 0: the contents of both strings are equal. ■ >0: the first character that does <u>not</u> match has a greater value in <i>puc1String</i> than in <i>puc2String</i>.

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

10.18 SAL_StrNCmp

This function compares num characters of the C string *puc1String* to mum characters of the C string *puc2String*. This function starts comparing the first character of each string.

Declaration

```
SALRetCode_t SAL_StrNCmp
(
    const uint8 * puc1String,
    const uint8 * puc2String,
    SALSize      uiLength,
    sint32 *     piRetCmp
);
```

Parameters

Value	Description
<i>puc1String</i>	[in] C string to be compared
<i>puc2String</i>	[in] C string to be compared
<i>uiLength</i>	[in] Character Length of the string to be compared. Refer to SALSize in Primitive Types .
<i>piRetCmp</i>	[out] Integral value indicating the relationship between the strings. ■ <0: the first character that does <u>not</u> match has a lower value in <i>puc1String</i> than in <i>puc2String</i>. ■ 0: the contents of both strings are equal. ■ >0: the first character that does <u>not</u> match has a greater value in <i>puc1String</i> than in <i>puc2String</i>.

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

10.19 SAL_StrLength

The length of the C string is determined by terminating null character: C string is as long as the number of characters between the beginning of string and the null-terminated string (Not including the null-terminate string itself).

Declaration

```
SALRetCode_t SAL_StrLength  
(  
    const uint8 * pucString,  
    SALSize *     puLength  
);
```

Parameters

Value	Description
<i>pucString</i>	<i>[in] C string</i>
<i>puLength</i>	<i>[out] Length of string. Refer to SALSize in the Primitive Types.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.20 SAL_QueueCreate

This function is called by user application to create a message queue. To use message queue, you must create it first.

Declaration

```
SALRetCode_t SAL_QueueCreate
(
    uint32 *    puiId,
    const uint8 * pucName,
    uint32      uiDepth,
    SALSize     uiDataSize
);
```

Parameters

Value	Description
<i>puiId</i>	<i>[out]</i> Identifier in Queue table
<i>pucName</i>	<i>[in]</i> Pointer to an ASCII string that will be used to name the message queue.
<i>uiDepth</i>	<i>[in]</i> The number of items the queue can hold. uiDepth indicates the maximum size of the message queue (must be non-zero).
<i>uiDataSize</i>	<i>[in]</i> Size of each item in the queue. Refer to SALSize in the Primitive Types .

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.21 SAL_QueueDelete

This function deletes a queue—freeing all the memory allocated for storing items placed in the queue.

Declaration

```
SALRetCode_t SAL_QueueDelete
(
    uint32    uiId
);
```

Parameters

Value	Description
uiId	[in] Identifier in Queue table to be deleted

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.22 SAL_QueueGet

This function waits for a message to be sent to the queue and receives an item from the queue. The item is copied to pData and received, so a buffer of adequate size must be provided. The number of bytes copied into the buffer is defined when the queue is created.

Declaration

```
SALRetCode_t SAL_QueueGet
(
    uint32          uiId,
    void *          pData,
    uint32 *        puiSizeCopied,
    sint32          iTimeout,
    SALBlockingOption_t ulOptions
);
```

Parameters

Value	Description
uiId	<i>[in] Identifier in Queue table</i>
pData	<i>[out] Pointer to the buffer into which the received item will be copied.</i>
puiSizeCopied	<i>[out] Pointer to a variable that will receive the size of the message</i>
iTimeout	<i>[in] The maximum amount of time that the task should be blocked. Queue should be empty at the time of the call.</i>
ulOptions	<i>[in] Blocking - depends on iTimeout. Non-blocking - is to be returned immediately. Refer to SALBlockingOption_t.</i>

Return

Value	Description
SAL_RET_SUCCESS	<i>Success</i>
SAL_RET_FAILED	<i>Failure</i>

Remark

None

10.23 SAL_QueuePut

This function posts an item to the queue. The item is queued by copy, not by reference. The item is normally posted to the queue (FIFO) at the end of the queue.

Declaration

```
SALRetCode_t SAL_QueuePut
(
    uint32          uiId,
    void *          pData,
    SALSize         uiSize,
    sint32          iTimeout,
    SALBlockingOption_t  ulOptions
);
```

Parameters

Value	Description
<i>uiId</i>	<i>[in] Identifier in Queue table</i>
<i>pData</i>	<i>[in] Pointer to the message to be sent</i>
<i>uiSize</i>	<i>[in] Size of the message (in byte)</i>
<i>iTimeout</i>	<i>[in] The maximum amount of time that the task should blocked. The task waits for space to become available on the queue, It means that the queue is already full. The call will return immediately if this is set to 0 and the queue is full.</i>
<i>ulOptions</i>	<i>[in] Blocking - depends on iTimeout. Non-blocking - is to be called immediately. Refer to SALBlockingOption t.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.24 SAL_EventCreate

This function is called to create an event group.

Declaration

```
SALRetCode_t SAL_EventCreate
(
    uint32 *      puiId,
    const uint8 *  pucName,
    uint32        uiInitialValue
);
```

Parameters

Value	Description
puiId	[out] Identifier in event table
pucName	[in] Pointer to the name of the event group
uiInitialValue	[in] Contains the initial value to be stored in the event group (typically 0).

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.25 SAL_EventDelete

This function deletes an event group and ensures that all tasks pending in the event group are ready.

Declaration

```
SALRetCode_t SAL_EventDelete  
(  
    uint32    uiId  
);
```

Parameters

Value	Description
<i>uiId</i>	<i>[in]</i> Identifier in event table

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.26 SAL_EventSet

This function is called to create an event group.

Declaration

```
SALRetCode_t  SAL_EventSet
(
    uint32      uiId,
    uint32      uiEvent,
    SALEventOption_t  ulOptions
);
```

Parameters

Value	Description
uiId	[in] Identifier in event table
uiEvent	[in] A bitwise value that indicates the bit or bits to set. For example, to set bit 3 only, set uiEvent to 0x08. To set bit 3 and bit 0, set uiEvent to 0x09.
ulOptions	[in] Indicates whether the flags will be set or cleared. Refer to SALEventOption t .

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.27 SAL_EventGet

This function is called to wait until the combination of bits in an event group is set. The application can wait for ANY bit to be set or for ALL bits to be set. This function cannot be called from an interrupt.

Declaration

```
SALRetCode_t SAL_EventGet
(
    uint32    uiId,
    uint32    uiEvent,
    uint32    iTimeout,
    uint32    uiOptions,
    uint32 *   puiFlags
);
```

Parameters

Value	Description
<i>uiId</i>	<i>[in]</i> Identifier in event table.
<i>uiEvent</i>	<i>[in]</i> A bitwise value that indicates the bit or bits to be tested inside the event group. For example, if your application requires to wait for bits 0 and 1, the 'flags' contains 0x03.
<i>iTimeout</i>	<i>[in]</i> The maximum amount of time (specified in 'ticks') to wait for one/all (depending on the <i>iTimeout</i> value) of the bits specified by <i>iTimeout</i> to be set.
<i>uiOptions</i>	<i>[in]</i> Specifies whether to set ALL bits or ANY bits.
<i>puiFlags</i>	<i>[out]</i> The flags in the event group that made the task ready, or 0 if being waited for becoming set or the block time expired.

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.28 SAL_MutexCreate

This function creates a mutex but it is not yet supported by RTOS. To use this function, you should implement this function in `sal_[freertos/ucos/...]_impl.c`

Declaration

```
SALRetCode_t SAL_MutexCreate  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.29 SAL_MutexDelete

This function deletes a mutex, but it is not yet supported by RTOS. To use this function, you should implement this function in `sal_[freertos/ucos/...]_impl.c`.

Declaration

```
SALRetCode_t SAL_MutexDelete  
(  
    uint32    uiId  
);
```

Parameters

Value	Description
<i>uiId</i>	<i>[in]</i> Identifier to be deleted in mutex table

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.30 SAL_MutexLock

The calling thread locks the mutex, but it is not yet supported by RTOS. To use this function, you should implement this function in sal_[freertos/ucos/...]_impl.c.

Declaration

```
SALRetCode_t SAL_MutexLock  
(  
    uint32    uiId,  
    uint32    uiTimeoutmsec  
);
```

Parameters

Value	Description
uiId	[in] Identifier to be locked in mutex table
uiTimeoutmsec	[in] Millisecond to be delayed

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.31 SAL_MutexUnlock

The calling thread unlocks the mutex and releases ownership. This function is not yes supported by RTOS. To use this function, you should implement this function in sal_[freertos/ucos/...]_impl.c.

Declaration

```
SALRetCode_t SAL_MutexUnlock  
(  
    uint32    uiId  
);
```

Parameters

Value	Description
uiId	[in] Identifier to be unlocked in mutex table

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.32 SAL_SemaphoreCreate

This function creates a semaphore.

Declaration

```
SALRetCode_t SAL_SemaphoreCreate
(
    uint32 *          puiId,
    const uint8 *      pucName,
    uint32             uiInitialValue,
    SALBlockingOption_t ulOptions
);
```

Parameters

Value	Description
<i>puiId</i>	<i>[out] Identifier in semaphore table</i>
<i>pucName</i>	<i>[in] Pointer to the name that may be given to the semaphore.</i>
<i>uiInitialValue</i>	<i>[in] Initial value for the semaphore. If this value is used to share resources, you should set the value to the number of resources available. If this value is used to signal the occurrence of event(s), then you should set the value to 0.</i>
<i>ulOptions</i>	<i>[in] Determines whether to block or not for using the semaphore. Refer to SALBlockingOption_t.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.33 SAL_SemaphoreDelete

This function deletes a semaphore.

Declaration

```
SALRetCode_t SAL_SemaphoreDelete
(
    uint32      uiId
);
```

Parameters

Value	Description
uiId	[in] Identifier to be deleted in semaphore table

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.34 SAL_SemaphoreRelease

This function signals/releases a semaphore.

Declaration

```
SALRetCode_t SAL_SemaphoreRelease
(
    uint32    uiId
);
```

Parameters

Value	Description
uiId	[in] Identifier to be released in semaphore table

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.35 SAL_SemaphoreWait

This function obtains/waits for a semaphore.

Declaration

```
SALRetCode_t SAL_SemaphoreWait
(
    uint32          uiId,
    sint32          iTimeout,
    SALBlockingOption_t  ulOptions
);
```

Parameters

Value	Description
uiId	[out] Identifier in semaphore table
iTimeout	[in] The time in ticks to wait for the semaphore to become available. A time of zero can be used to poll the semaphore. If non-zero, your task will wait for the resource up to the amount of time (in 'ticks') specified by this argument.
ulOptions	[in] Determines whether to block or not for using the semaphore. Refer to SALBlockingOption_t .

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.36 SAL_TaskCreate

This function is used to have an OS for managing task execution. You can create a task before you start multitasking or from a running task. However, you cannot create a task in Interrupt Service Routine (ISR).

Declaration

```
SALRetCode_t SAL_TaskCreate
(
    uint32 *          puiTaskId,
    const uint8 *     pucTaskName,
    SALTaskFunc       fnTask,
    uint32 *          puiStackPtr,
    uint32            uiStackSize,
    SALTaskPriority_t uiPriority,
    void *            pTaskParam
);
```

Parameters

Value	Description
<i>puiTaskId</i>	[out] Pointer to identifier in task table
<i>pucTaskName</i>	[in] Pointer to an ASCII string to provide a name to the task.
<i>fnTask</i>	[in] Pointer to the task's code, that is a function pointer. Refer to SALTaskFunc .
<i>puiStackPtr</i>	[out] Pointer to the base address of the stack (i.e. low address).
<i>uiStackSize</i>	[in] Stack size of the number of elements.
<i>uiPriority</i>	[in] The task's priority. A unique priority MUST be assigned to each task and the lower the number, the higher the priority. Refer to SALTaskPriority_t .
<i>pTaskParam</i>	[in] Pointer to an optional data area which can be used to pass parameters to the task when the task is executed first.

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

10.37 SAL_TaskDelete

This function is called to remove a task from the RTOS real time management. The task being deleted is removed from all ready, blocked, suspended, and event lists.

Declaration

```
SALRetCode_t SAL_TaskDelete  
(  
    uint32    uiTaskId  
);
```

Parameters

Value	Description
<i>uiTaskId</i>	<i>[in] Identifier in task table</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.38 SAL_TaskCurrent

This function is called to know a current running task, but it is not yet supported by RTOS. To use this function, you should implement this function in sal_[freertos/ucos/...]_impl.c.

Declaration

```
SALRetCode_t SAL_TaskCurrent  
(  
    uint32 *   puiTaskId  
);
```

Parameters

Value	Description
<i>puiTaskId</i>	<i>[out] Identifier that you want to know from the task table.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.39 SAL_TaskSleep

This function is called to delay execution of the currently running task until the specified number of milliseconds expires. The actual time that the task remains blocked depends on the tick rate.

Declaration

```
SALRetCode_t SAL_TaskSleep  
(  
    uint32    uiMilliSec  
);
```

Parameters

Value	Description
<i>uiMilliSec</i>	<i>[in]</i> The amount of time, in milliseconds, that the calling task should block.

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

If the value of *uiMilliSec* is 1 ms, the system may or may not sleep for 1 ms. That is because the tick of OS system is basically 1 ms. Therefore, it is recommended to pass a *uiMilliSec* value greater than 1 ms as a parameter. For more information, contact Telechips. [2]

10.40 SAL_DbgReportError

This function is called to inform MCU system of a specific error.

Declaration

```
SALRetCode_t SAL_DbgReportError
(
    SALDriverId_t    uiDriverId,
    uint32           uiApiId,
    SALErrorCode_t   uiErrorCode,
    const int8 *      pucEtc
);
```

Parameters

Value	Description
<i>uiDriverId</i>	[in] Identifier of device driver or module in MCU BSP. Refer to SALDriverId t .
<i>uiApiId</i>	[in] Identifier of API functions
<i>uiErrorCode</i>	[in] Error code of each driver or module. Refer to SALErrorCode t .
<i>pucEtc</i>	[in] Additional error string

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

10.41 SAL_GetCurError

This function is called to get information of the detailed error that has been reported.

Declaration

```
SALRetCode_t SAL_GetCurError
(
    SLErrorInfo_t * psInfo
);
```

Parameters

Value	Description
psInfo	[out] Pointer to information of a currently reported error. Refer to SLErrorInfo_t .

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.42 SAL_GetTickCount

This function is used by your application to obtain the current value of the counter that keeps track of the number of clock ticks.

Declaration

```
SALRetCode_t SAL_GetTickCount
(
    uint32 *   puiTickCount
);
```

Parameters

Value	Description
puiTickCount	[out] Pointer to the current count of ticks

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.43 SAL_HandleTick

This function sets interrupt mask before altering scheduler structures. The tick handler runs at the lowest priority, so interrupts cannot already be masked. There is no need to save and restore the current mask value. It is necessary to turn off interrupts in the CPU itself while the Interrupt Priority Mask Register (ICCPMR) is being updated.

Declaration

```
SALRetCode_t SAL_HandleTick  
(  
    void  
);
```

Parameters

None

Return

<i>Value</i>	<i>Description</i>
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.44 SAL_GetSystemTickCount

This function is used by your application to obtain the current value of the counter that keeps track of the number of system ticks based on timer module.

Declaration

```
SALRetCode_t SAL_GetSystemTickCount  
(  
    uint64 *    pullSysTickCount  
);
```

Parameters

Value	Description
<i>pullSysTickCount</i>	<i>[out] Pointer to the current count of system ticks based on timer module.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

10.45 SAL_IncreaseSystemTick

This function is used by your application to increase the value of the counter that keeps track of the number of system ticks based on timer module.

Declaration

```
SALRetCode_t SAL_IncreaseSystemTick
(
    uint64 *    pullSysTickCount
);
```

Parameters

Value	Description
pullSysTickCount	[out] Pointer to the current count of system ticks based on timer module.

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

10.46 SAL_GetVersion

This function is to return the version information of MCU BSP.

Declaration

```
SALRetCode_t SAL_GetVersion  
(  
    SALMcuVersionInfo_t *    psVersion  
);
```

Parameters

Value	Description
<i>psVersion</i>	[out] Pointer to the version information of MCU BSP. Refer to SALMcuVersionInfo_t .

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	Success
<i>SAL_RET_FAILED</i>	Failure

Remark

None

11 HOW TO USE SAL

11.1 Register SAL APIs

The SAL APIs are registered by invoking **SAL_RegisterFunction()** in **sal_api.h**.

```
{
    SALRetCode_t UC_QueueGet(uint32 uiId, void * pData, SALSize uiSize, uint32 * puiSizeCopied,
    sint32 iTimeout, SALBlockingOption_t ulOptions)
    {
        SALRetCode_t  retval          = SAL_RET_SUCCESS;
        OS_ERR        err;
        CPU_TS        ts;
        OS_MSG_SIZE    msg_copied     = 0;
        CPU_INT08U *   queue_data;

        if ((gQueueObj[uiId].qiUsed == TRUE) && (pData != NULL) && (puiSizeCopied != NULL))
        {
            queue_data = OSQPend((OS_Q *)&gQueueObj[uiId].qiCb,
                                (OS_TICK)iTimeout,
                                (OS_OPT)OS_OPT_PEND_BLOCKING,
                                (OS_MSG_SIZE *)&msg_copied,
                                (CPU_TS *)&ts,
                                (OS_ERR *)&err);

            if ((err == OS_ERR_NONE) && (queue_data != 0) && (msg_copied > 0))
            {
                UC_MemCopy(pData, queue_data, msg_copied);
                *puiSizeCopied = (uint32)msg_copied;
            }
            else
            {
                retval = SAL_RET_FAILED;
                UC_ReportError(SAL_DRVID_SAL, SAL_ERR_FAIL_GET_DATA, __FUNCTION__);
            }
        }
        else
        {
            retval = SAL_RET_FAILED;
            UC_ReportError(SAL_DRVID_SAL, SAL_ERR_INVALID_PARAMETER, __FUNCTION__);
        }
    }

    return retval;
}

SALRetCode_t SAL_Init (void)
{
    ...
    SAL_RegisterFunction(SAL_REG_DATA_QUEUE_GET, &UC_QueueGet, NULL_PTR);
    ...
}
}
```

11.2 Initialize SAL

The SAL is initialized by using **SAL_Init()** in **sal_api.h**. The **SAL_Init()** is composed of initialization codes for OS itself and SAL object as sample code below in case of uC/OS-III version.

```
SALRetCode_t SAL_Init (void)
{
    ...
    SAL_RegisterFunction(SAL_REG_CORE_MB, &UC_CoreMB, NULL_PTR);
    SAL_RegisterFunction(SAL_REG_CORE_CRITICAL_ENTER, &UC_CoreCriticalEnter, NULL_PTR);
    SAL_RegisterFunction(SAL_REG_CORE_CRITICAL_EXIT, &UC_CoreCriticalExit, NULL_PTR);
    ...

    /* Init uC/OS-III */
    OSInit(&err);

    /*OASL default manage object*/
    OSSemCreate((OS_SEM *)&gSemObj,
                (CPU_CHAR *)"OSAL Sema",
                (OS_SEM_CTR)1U,
                (OS_ERR *)&err);
}
```

11.3 Start OS by using SAL

The **SAL_OsStart()** API in **sal_internal.h** starts OS services including OS scheduler. For this, **SAL_OsStart()** can be written as sample implementation below based on uC/OS-III version.

```
SALRetCode_t UC_OSStart (void)
{
    OS_ERR      err;
    SALRetCode_t retval = SAL_RET_SUCCESS;

    /* Start multitasking (i.e. give control to uC/OS-III). */
    OSStart(&err);

    if (err != OS_ERR_NONE)
    {
        retval = SAL_RET_FAILED;
        UC_ReportError(SAL_DRVID_SAL, SAL_ERR_FAIL_START, __FUNCTION__);
    }

    return retval;
}
```

There are some optional functions that are supported by only specific OS. For this, the SAL supports the **SAL_OsInitFuncs()** API in **sal_internal.h**. The following code shows an example for this API based on the uC/OS-III version.

```

SALRetCode_t UC_OSInitFuncs (void)
{
    #if (OS_CFG_STAT_TASK_EN > 0U)
        OS_ERR err;
    #endif

    SALRetCode_t  retval = SAL_RET_SUCCESS;

    (void)OS_TmrTickInit();
    CPU_Init();          /* Initialize the uC/CPU services          */

    #if (OS_CFG_STAT_TASK_EN > 0U)
        OSStatTaskCPUUsageInit(&err);    /* Compute CPU capacity with no task running */

        if (err != OS_ERR_NONE)
        {
            retval = SAL_RET_FAILED;
            UC_ReportError(SAL_DRVID_SAL, SAL_ERR_INIT, __FUNCTION__);
        }
    #endif

    #ifdef CPU_CFG_INT_DIS_MEAS_EN
        CPU_IntDisMeasMaxCurReset();
    #endif

    return retval;
}

```

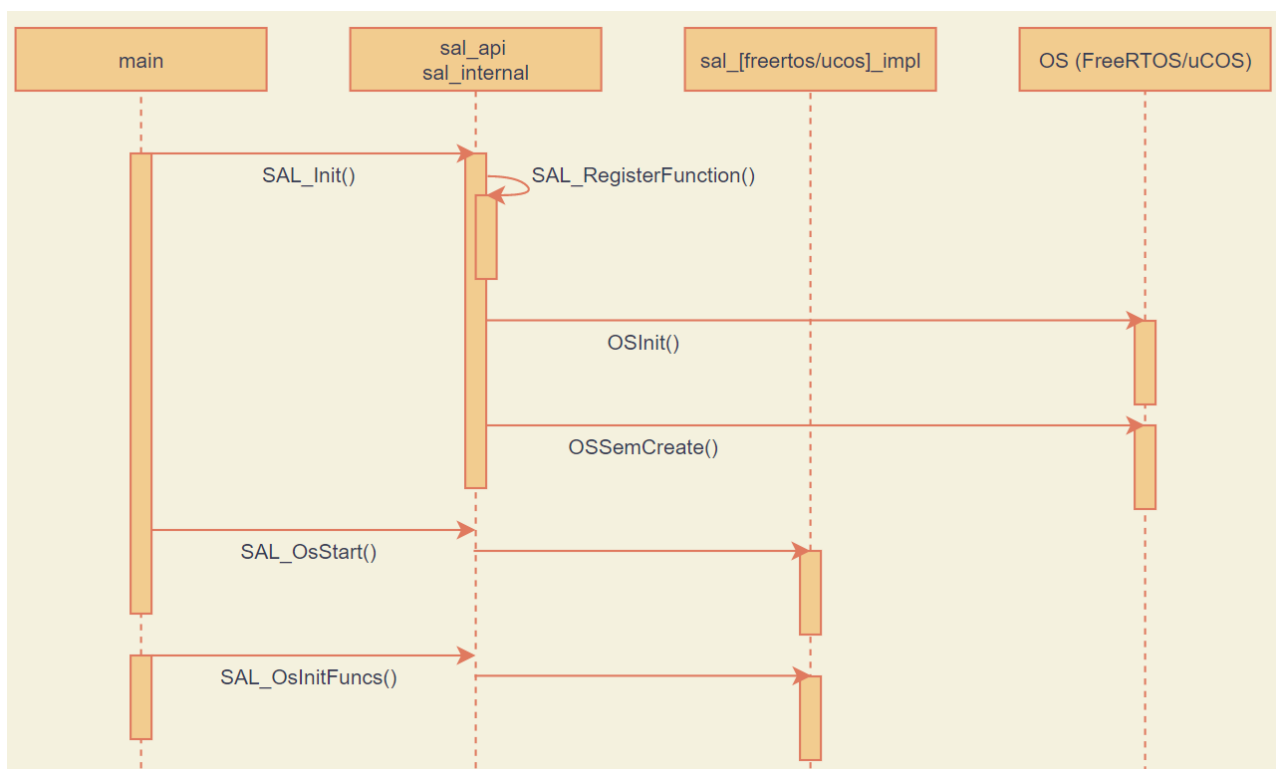


Figure 11.1 Flowchart of Registering/Initializing SAL APIs

11.4 Report and Handle Error

All SAL APIs return only information of success or failure. Since information on the current status is stored in the structure of *SALErrorInfo_t*, detailed error information can be figured through this structure. As soon as *SAL_GetCurError()* is called and the current error is obtained, error information is destroyed and cannot be obtained again. In other words, error information is retained until *SAL_GetCurError()* is invoked. In addition, the error structure with error information is updated through *SAL_DbgReportError()*.

SAL_DbgReportError() can be written as sample implementation as follows.

```
{
    SALRetCode_t  ret    = SAL_RET_SUCCESS;

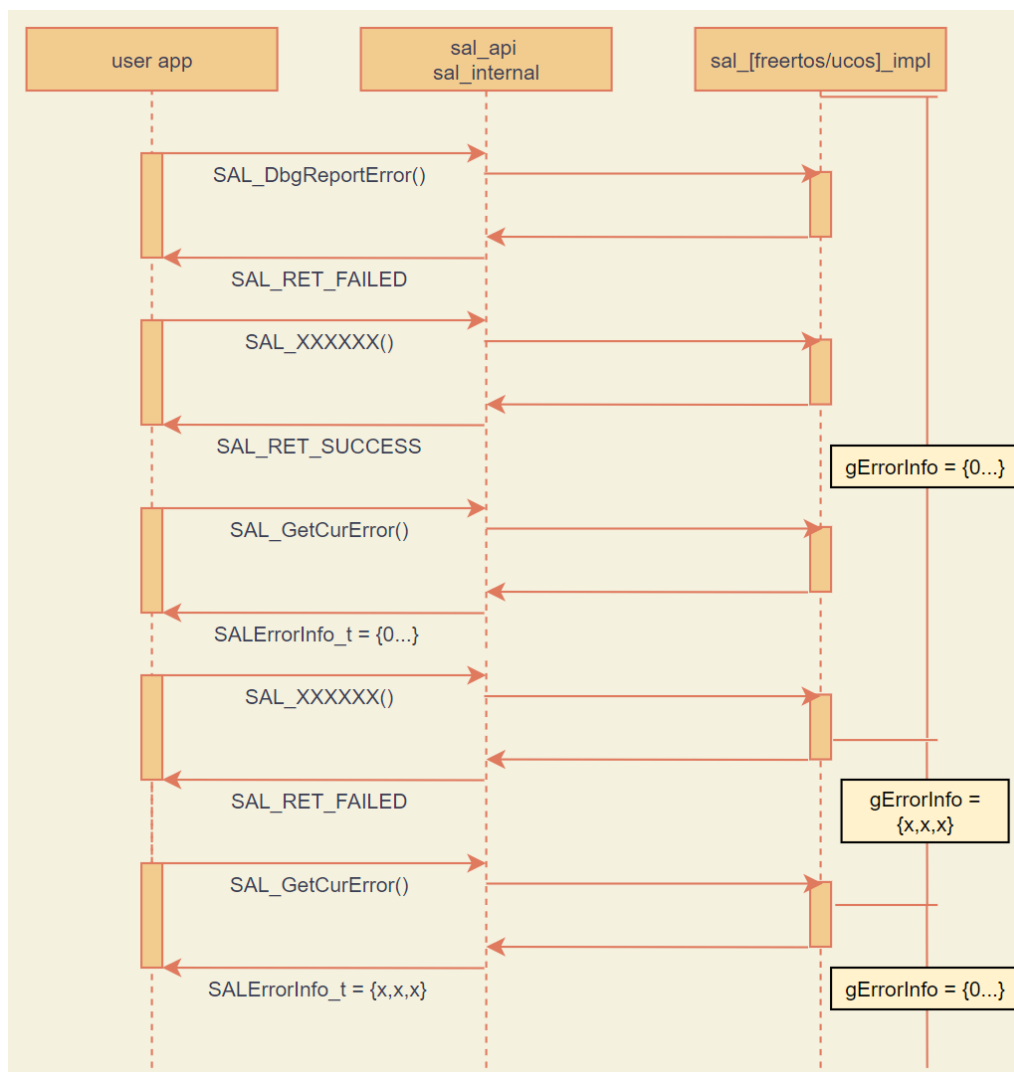
    ret = SAL_DbgReportError(SAL_DRVID_TMR,
                            TIMER_API_ENABLE_CFG,
                            TIMER_ERR_NOT_INITIALIZED,
                            __FUNCTION__);
}
```

SAL_GetCurError() can be written as sample implementation as follows.

```
{
    SALErrorInfo_t  errorTable;
    SALRetCode_t    err;

    err = SAL_TaskCreate(&AppTaskStartID,
                        (const uint8 *)"App Task Start",
                        (SALTaskFunc)Main_StartTask,
                        &AppTaskStartStk[0],
                        ACFG_TASK_MEDIUM_STK_SIZE,
                        SAL_PRIO_APP_CFG,
                        NULL);

    if (err == SAL_RET_FAILED)
    {
        SAL_GetCurError(&errorTable);
    }
}
```

**Figure 11.2 Flowchart of Reporting/Handling an Error**

12 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

13 REVISION HISTORY

Rev. 1.02: 2021-06-01

- Updated
 - Chapter 6.5: *SAL_EVENT_OPT_CONSUME* description
 - Chapter 10.27: *puiFlags* description
 - Chapter 10.39: added Remark

Rev. 1.01: 2021-01-22

- Updated
 - Chapter 5.3: Declaration

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."